Compactação de dados

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Algoritmo da frequência de caracteres
- Algoritmo de Huffman
- Exercícios

Introdução

- Compactação de dados: codificação de dados de visando minimizar espaço utilizado
  - Problema crítico nos primórdios da computação: dispositivos muito limitados
  - Atualmente é muito útil:
    - Algumas aplicações exigem quantidade muito grande de dados
    - Redução do custo de transmissão em redes
  - Necessidade de algoritmos para codificação/decodificação de dados

Algoritmo da frequência de caracteres

- Algoritmo da frequência de caracteres: sequência de símbolos idênticos é substituída por código
  - Código: quantidade e símbolo
- Exemplo de texto:

$$BBBEAAAAFFHHHHHCBMMALLLCDDBBBBBBBCC$$

  - Texto compactado: $3B1E4A2F5H1C1B2M1A3L1C2D7B2C$

- Melhoria: não utilizar quantidade quando há 1 símbolo
  - Texto compactado: $3BE4A2F5HCB2MA3LC2D7B2C$

Algoritmo da frequência de caracteres

- Dificuldade: não há como representar símbolos numéricos
  - Solução: utilizar símbolo especial para indicar que número a seguir é um símbolo
    - Símbolo especial não pode aparecer no texto!
- Exemplo de texto:

$$AAA33333BA6666888DDDDDDD99999999999AABBB$$

  - Texto compactado: $3A5@3BA4@63@87D11@92A3B$
    - Símbolo especial '@' não pode ocorrer no texto

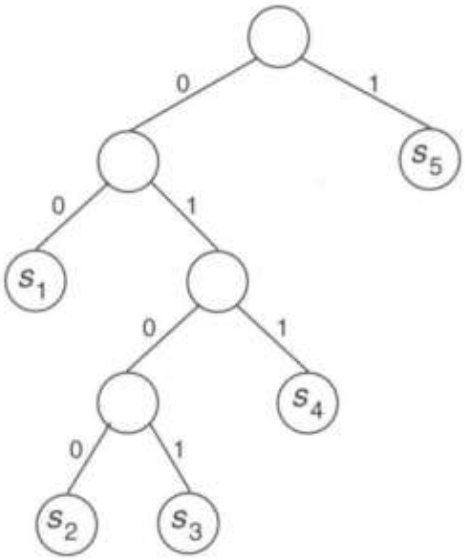
Algoritmo de Huffman

- Algoritmo de Huffman: realiza codificação de prefixos de forma ótima
- Seja que o texto é formado por um conjunto de símbolos $S = \{s_1, ..., s_n\}$, $n > 1$
  - Sabe-se a frequência f_i de cada símbolo s_i no texto
  - Deseja-se atribuir um código a cada símbolo para compactar o texto
  - Nenhum código é prefixo de outro: árvore estritamente binária de prefixo
    - Cada símbolo s_i está associado a uma folha
    - Código de s_i: caminho da raiz até a folha correspondente a s_i

Algoritmo de Huffman

- Exemplo de códigos de prefixo:

símbolo	*código*
s_1	00
s_2	0100
s_3	0101
s_4	011
s_5	1



- Exemplo de texto: $s_4\ s_3\ s_3\ s_1\ s_3\ s_1\ s_4\ s_5\ s_1\ s_3\ s_3\ s_3\ s_3\ s_3\ s_2\ s_3\ s_5\ s_2\ s_2\ s_2\ s_4$
- Texto codificado:

011010101010001010001110001010101010101010101010001011010001000100011

Algoritmo de Huffman

- Vantagem do algoritmo: facilidade de codificação/decodificação
  - Codificação: utilização de tabela de códigos construída através da árvore
  - Decodificação: percorrer texto da esquerda para direita realizando percursos iniciando na raiz da árvore
    - Ao chegar numa folha, um símbolo é decodificado

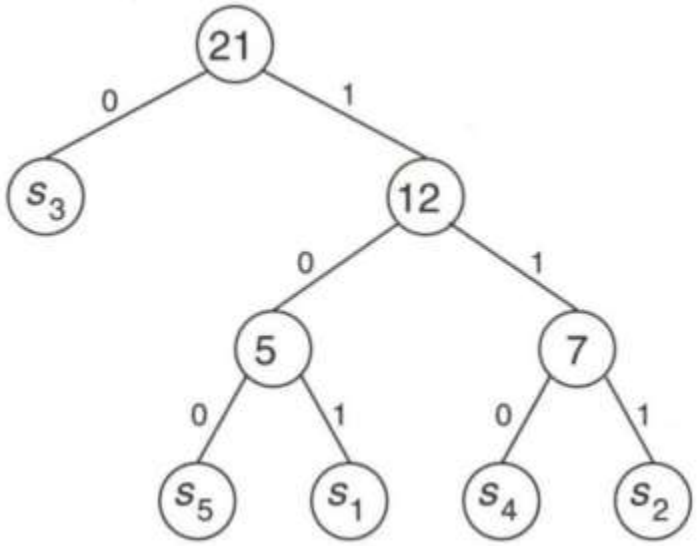
Algoritmo de Huffman

- Qual o comprimento do texto codificado?
  - Custo de T ou c(T): comprimento da sequência binária produzida pela árvore binária de prefixo T
    - No exemplo anterior: custo da árvore é 69
- Árvore de Huffman possui custo mínimo

Algoritmo de Huffman

- Exemplo: texto, árvore de Huffman e texto codificado
  - Texto a ser codificado: $s_4\ s_3\ s_3\ s_1\ s_3\ s_1\ s_4\ s_5\ s_1\ s_3\ s_3\ s_3\ s_3\ s_3\ s_2\ s_3\ s_5\ s_2\ s_2\ s_2\ s_4$
  - Árvore de Huffman:

símbolo	*freqüência*	*código*
s_1	3	101
s_2	4	111
s_3	9	1
s_4	3	110
s_5	2	100



  - Texto codificado: 110001010101110100101000001110100111111111110
    - Custo da árvore: 45

Algoritmo de Huffman

- Seja T uma árvore de prefixo correspondente a um texto onde cada símbolo s_i ocorre f_i vezes
  - Denote l_i o comprimento do código binário de s_i
  - Cada símbolo s_i contribui com $f_i l_i$ unidades de custo c(T)
  - $c(T) = \sum_{1 \leq i \leq n} f_i l_i$

Algoritmo de Huffman

- Características do algoritmo de Huffman:
  - Utiliza estratégia gulosa
  - Constrói árvore mínima de forma iterativa
  - Árvore é construída das folhas para a raiz
    - São obtidos subcódigos para subconjuntos de símbolos
  - Processo iterativo envolve fusões de subárvores
  - Algoritmo termina quando só há uma subárvore

Algoritmo de Huffman

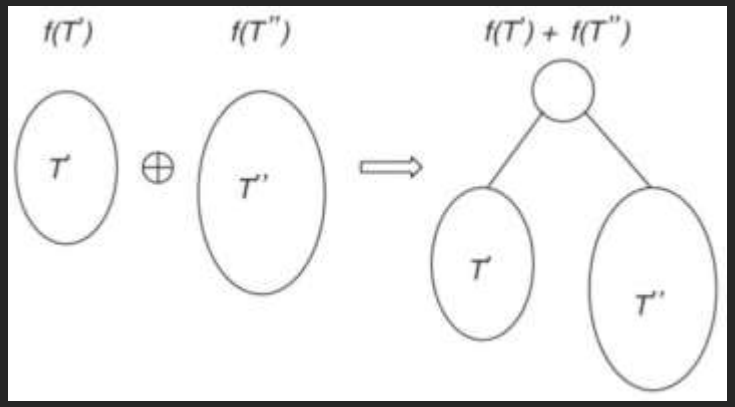
- Seja T’ uma subárvore binária de prefixo:
  - As folhas de T’ são símbolos dentre $\{s_1, ..., s_n\}$
  - Define-se f(T’) a frequência de T’: soma das frequências dos símbolos s_i presentes nas folhas de T’
- Nós internos de uma árvore de Huffman: frequência das subárvores de raiz com raiz nesses nós

Algoritmo de Huffman

- Funcionamento do algoritmo para construção da árvore de Huffman:
  - Inicialmente há n subárvores, uma para cada símbolo s_i
    - Frequência de cada subárvore = frequência do símbolo
  - Iterativamente:
    - Selecione duas subárvores T’ e T’’, tais que f(T’) e f(T’’) são mínimas
    - T’ e T’’ são fundidas em uma única árvore T: operação $\oplus$
  - Algoritmo termina quando resta apenas 1 subárvore: n-1 fusões

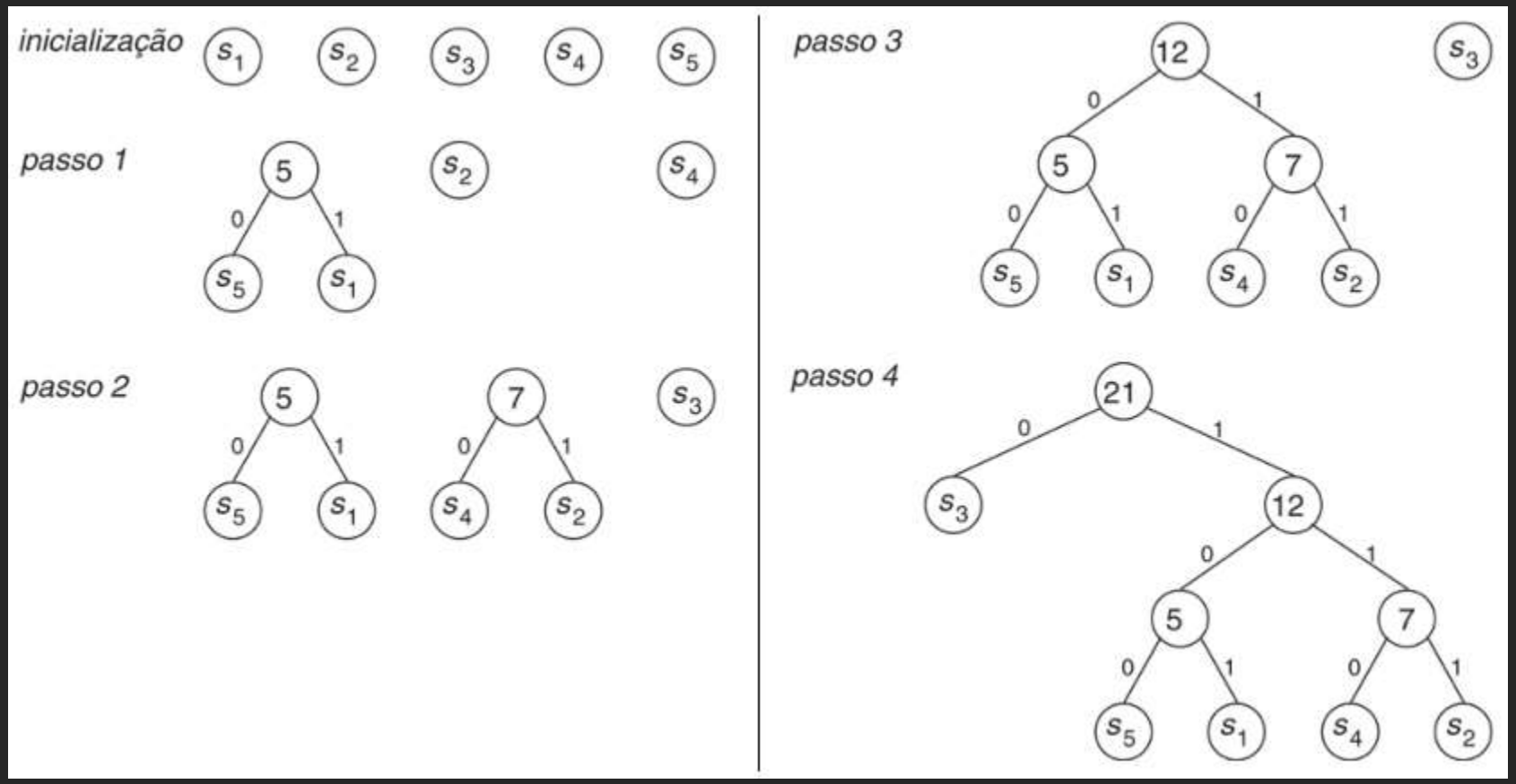
Algoritmo de Huffman

- Operação de fusão T = T’ ⊕ T’’:
  - É criado um novo nó com filhos esquerdo e direito sendo as raízes de T’ e T’’
    - É indiferente se T’ é filho direito ou esquerdo
  - Esse nó é a raiz de T
  - A frequência f(T) = f(T’ ⊕ T’’) = f(T’) + f(T’’)
- Ilustração da operação ⊕:



Algoritmo de Huffman

- Exemplo de construção da árvore de Huffman:



Algoritmo de Huffman

- Algoritmo de Huffman:
  - F é uma lista de prioridades mínima
  - minimo(T’, f, F): retira menor prioridade (f) e rearranja F
  - inserir(T, f, F): inclui T de prioridade f na lista F

algoritmo 10.4: construção da árvore de Huffman

para $i := 1, n-1$ **faça**

$$minimo(T', f, F); \quad minimo(T'', f, F)$$

$$T := T' \oplus T''$$

$$f(T) := f(T') + f(T'')$$

$$inserir(T, f, F)$$

Algoritmo de Huffman

- Complexidade de tempo: O(n lg n)
  - n-1 iterações
  - Cada operação de inserção e remoção em lista de prioridade pode ser feita em O(lg n)
  - Cada operação $\oplus$ gasta O(1)
- Problema do código de Huffman: um erro de transmissão em apenas um dígito (bit) prejudica toda a decodificação

Exercícios

- 10.14 a 10.16
- 10.18